

How the Code Works

This project implements a preemptive multithreading package for the 8051 microcontroller. Unlike the cooperative version, threads do not need to manually call `ThreadYield()` to give up control. Instead, Timer 0 is configured in `Bootstrap()` to generate periodic interrupts. Each Timer 0 interrupt jumps to `myTimer0Handler()`, which performs the context switch automatically.

The thread package uses manually allocated internal RAM to store the saved stack pointer for each thread, the current thread ID, and the active-thread bitmap. Each thread has its own 16-byte stack region and uses a separate 8051 register bank. Thread 0 uses register bank 0, thread 1 uses register bank 1, and so on. This allows the program to avoid saving and restoring registers R0 through R7 for every thread.

When `Bootstrap()` runs, it initializes the thread-management variables, creates `main()` as the first thread, enables the Timer 0 interrupt, starts Timer 0, and restores the context of the first thread. After this, `main()` creates the Producer thread and then runs the Consumer function.

`ThreadCreate()` is responsible for creating a new thread. It finds an unused thread ID, marks it as active in the bitmap, assigns the correct stack region, and pushes the starting function address onto the new stack. It also initializes the saved register context, including ACC, B, DPL, DPH, R0, and PSW. The PSW value selects the register bank assigned to the thread. After the initial stack is prepared, the stack pointer is saved in the `savedSP` array.

Context switching is handled by `myTimer0Handler()`. When Timer 0 interrupts the current thread, the handler saves the current thread's registers and stack pointer. It then selects the next valid thread using a round-robin policy by checking the thread bitmap. After selecting the next thread, it restores that thread's saved stack pointer and registers, then returns from the interrupt using `RETI`. This causes execution to continue in the selected thread instead of the interrupted one.

The test program uses a single-buffer producer-consumer example. The Producer generates characters from `'A'` to `'Z'` repeatedly and writes one character at a time into `sharedBuffer`. The Consumer waits until the buffer is full, reads the character, sends it to the UART through `SBUF`, and marks the buffer as empty again. The variable `bufferFull` is used to indicate whether the shared buffer currently contains valid data.

Because the Producer and Consumer share `sharedBuffer` and `bufferFull`, interrupts are temporarily disabled using `EA = 0` while these shared variables are being checked or updated.

This prevents Timer 0 from preempting a thread in the middle of a shared-buffer operation. After the critical section is complete, interrupts are re-enabled with `EA = 1`.

When the program runs correctly in EdSim51, the UART receive window displays repeating letters from `A` to `Z`. This shows that the Producer is generating characters, the Consumer is transmitting them, and Timer 0 is preemptively switching between the two threads.

3.1 Screenshots for compilation

```
PS C:\Users\Theresia\Downloads\112006221_ppc2> make clean
powershell -NoProfile -Command "Remove-Item -ErrorAction SilentlyContinue *.hex,*.ihx,*.lnk,*.lst,*.map,*.mem,*.rel,*.rst,*.sym,*.asm,*.lk"
PS C:\Users\Theresia\Downloads\112006221_ppc2> make
sdcc -c testpreempt.c
sdcc -c preemptive.c
sdcc -o testpreempt.hex testpreempt.rel preemptive.rel
PS C:\Users\Theresia\Downloads\112006221_ppc2> Get-ChildItem
```

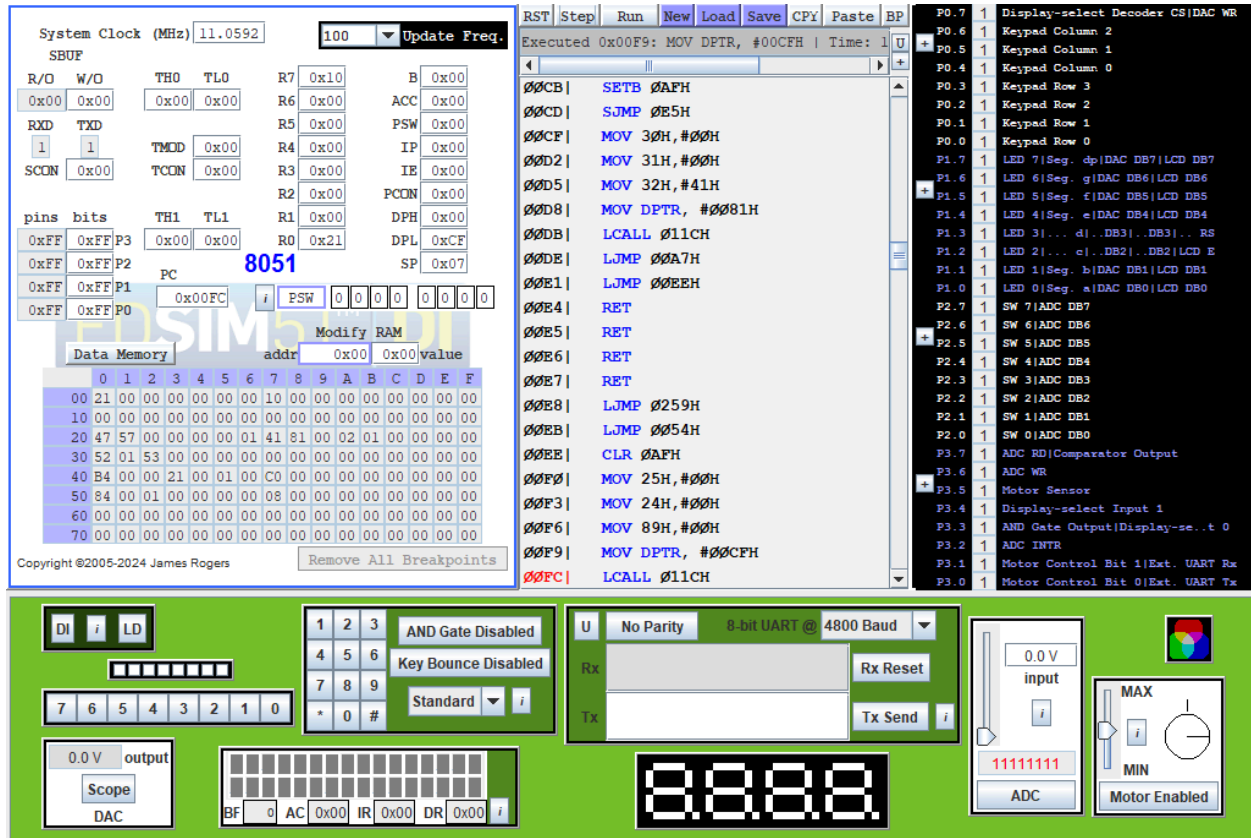
Directory: C:\Users\Theresia\Downloads\112006221_ppc2

Mode	LastWriteTime		Length	Name
----	-----		-----	----
-a----	6/13/2026	6:25 AM	559	Makefile
-a----	6/13/2026	6:51 AM	14658	preemptive.asm
-a----	6/13/2026	6:58 AM	7396	preemptive.c
-a----	6/13/2026	6:20 AM	289	preemptive.h
-a----	6/13/2026	6:51 AM	40164	preemptive.lst
-a----	6/13/2026	6:51 AM	6265	preemptive.rel
-a----	6/13/2026	6:51 AM	40818	preemptive.rst
-a----	6/13/2026	6:51 AM	40876	preemptive.sym
-a----	6/13/2026	6:51 AM	14125	testpreempt.asm
-a----	6/13/2026	6:48 AM	5001	testpreempt.c
-a----	6/13/2026	6:51 AM	1735	testpreempt.hex
-a----	6/13/2026	6:51 AM	261	testpreempt.lk
-a----	6/13/2026	6:51 AM	35887	testpreempt.lst
-a----	6/13/2026	6:51 AM	17343	testpreempt.map
-a----	6/13/2026	6:51 AM	1192	testpreempt.mem
-a----	6/13/2026	6:51 AM	5540	testpreempt.rel
-a----	6/13/2026	6:51 AM	36445	testpreempt.rst
-a----	6/13/2026	6:51 AM	40315	testpreempt.sym

```
PS C:\Users\Theresia\Downloads\112006221_ppc2>
```

3.2

Part 1: Before ThreadCreate call



This screenshot was taken immediately before Bootstrap() calls ThreadCreate(main). At this point, no thread has been created yet, so threadBitmap at address 0x25 is 0x00, and currentThread at address 0x24 is 0x00. The stack pointer is still at 0x07, which is the default startup stack location. During ThreadCreate(main), thread 0 is allocated, a new stack is prepared for it starting at address 0x40, the address of main() is pushed onto the stack, and the initial register context is added. The final stack pointer is then stored in savedSP[0], and the thread bitmap becomes 0x01.

Part 2: Before ThreadCreate(Producer)

The screenshot displays the DSIM5 microcontroller simulator interface. The top section shows the 'System Clock (MHz)' at 11.0592 and 'Update Freq.' at 100. Below this is the 'R/W' register window with various registers like TH0, TL0, R7, B, etc. The 'Data Memory' window shows a memory dump with addresses 0000 to 00FF. The 'Modify RAM' window is also visible. The 'Instructions' window displays assembly code starting with 'CLR 0AFH'. The 'I/O' window on the right shows various peripheral status bits like 'Keypad Column 2', 'Keypad Column 1', etc. The bottom section shows the 'Hardware' window with a keypad, a 4-digit display showing '8888', and various control buttons like 'AND Gate Disabled', 'Key Bounce Disabled', 'Standard', 'Rx Reset', 'Tx Send', 'Scope', 'DAC', 'ADC', and 'Motor Enabled'.

This screenshot was taken immediately before `main()` calls `ThreadCreate(Producer)`. At this point, thread 0 has already been created and is running `main()`, so `currentThread` at address 0x24 is 0x00. The thread bitmap at address 0x25 is 0x01, showing that only thread 0 is currently active. `DPTR` contains 0x0081, which is the address of `Producer()` and is passed to `ThreadCreate()` as the function pointer. During this call, thread ID 1 is allocated and the bitmap becomes 0x03. A new stack is prepared for thread 1 beginning at address 0x50. The address of `Producer()` and the initial register context are pushed onto this stack. The `PSW` value selects register bank 1, and the final stack pointer is saved in `savedSP[1]`.

Part 3: Producer Running

The screenshot displays the Proteus 8.10 SP3 IDE interface during a simulation. The main window shows assembly code for a PIC18F4550 microcontroller. The register window on the left indicates the Program Counter (PC) is at address 0x0094, and the PSW (Program Status Word) is 0x0000. The hardware simulation at the bottom shows a PIC18F4550 microcontroller connected to a keypad, a display, and various peripheral components like an ADC and a motor sensor. The display shows the character 'A'.

Assembly Code:

```

00091: MOV 31H, #01H
00092: ANL A, #07H
00093: CJNE A, #06H, 00094
00094: JNC 04H
00095: MOV A, @R0
00096: ANL A, #0F8H
00097: MOV @R0, A
00098: POP 08H
00099: POP 08H
00100: RETI
00101: LJMP 0051H
00102: MOV A, 31H
00103: JNZ 0FCH
00104: CLR 0AFH
00105: MOV A, 31H
00106: JNZ 15H
00107: MOV 30H, 32H
00108: MOV 31H, #01H
00109: MOV A, #5AH
  
```

Register Window:

R7	R6	R5	R4	R3	R2	R1	R0	B	ACC	PSW	IP	IE	PCON	DPH	DPL	SP
0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x08	0x00	0x02	0x00	0x00	0x00	0x4F

Hardware Simulation:

- Keypad:** A 4x4 matrix keypad is connected to the microcontroller.
- Display:** A 16x2 LCD display shows the character 'A'.
- ADC:** An ADC module is connected to the microcontroller.
- Motor Sensor:** A motor sensor module is connected to the microcontroller.

The Producer is running because the program counter is inside the Producer() function and currentThread at address 0x24 contains 0x01, meaning thread 1 is active. The PSW value is 0x08, which selects register bank 1 for thread 1. The shared buffer at address 0x30 contains 0x41, representing the character 'A', and bufferFull at address 0x31 is 0x01. This confirms that the Producer has generated a character and placed it into the shared buffer. Since this is the preemptive version, the Producer does not call ThreadYield(); Timer 0 performs the context switching automatically.

Part 4: Consumer Running

The screenshot displays the EdSim51 microcontroller simulator interface. The top section shows the system clock at 11.0592 MHz and the program counter (PC) at 8051. The register window on the left shows the status of various registers, including R0 through R7, ACC, PSW, IP, IE, PCON, DPH, DPL, and SP. The main window displays the assembly code being executed, with the current instruction being `MOV 31H, #00H`. The right-hand pane shows the I/O devices, including the Display-select Decoder, Keypad, and various ADCs. The bottom section shows the hardware components, including the DAC, UART, and Motor. The UART output shows the character 'A' being sent.

The Consumer is running because the program counter is inside the Consumer() function, near the instructions that send the shared character to the UART and clear the buffer flag. The current thread ID at address 0x24 is 0x00, indicating that thread 0 is active. The value in sharedBuffer at address 0x30 is 0x41, which represents 'A', and the same value has been copied into SBUF. The value of bufferFull at address 0x31 is now 0x00, showing that the Consumer has consumed the character and released the shared buffer for the Producer. Since this is the preemptive version, Timer 0 handles the switching between Producer and Consumer automatically.

Part 5: How can you tell that the interrupt is triggering on a regular basis?

The Timer 0 interrupt is triggered on a regular basis because the program continues switching between the Producer and Consumer even though neither function calls ThreadYield(). In EdSim51, Timer 0 is enabled because IE = 0x82, and Timer 0 is running as shown by the changing TH0 and TL0 values. The value of currentThread at address 0x24 changes between 0x00 and 0x01, showing that the interrupt handler is periodically saving the current thread state and restoring the next thread. The UART output also continues to print characters, confirming that both threads are being scheduled repeatedly by the timer interrupt.